Entangle, explained: a reading script for the team

For Nikola and Dejan. Read it once, top to bottom, before the demo and the jury questions. It is about 30 minutes. The code was written fast with an AI assistant, so this document does the other half of the job: it says what the code **does**, **why** it is built this way, and **where** to open it when someone asks.

Three ways to use it:

1. **Read Parts 1 to 3** (10 minutes) to be able to explain the idea and the architecture to anyone.
2. **Read Parts 4 to 8** when you want to answer "how does X work, exactly?" Each part ends with the file to open.
3. **Keep Parts 9 and 10 open** during questions: the honest limits, and the questions we expect with a short answer each.

Every number here comes from the committed reference run `out/golden/` (`summary.json`, `trace.jsonl`) or from quality.md. File names are relative to the repository root. A `function()` name is something you can search for.

Part 1. The idea in one page

The problem. EINSTEIN is a Horizon Europe project that connects the University of Novi Sad with companies around NTP. Someone has to read 153 company profiles (489 challenges) and 1,047 researcher profiles with 10,779 publications, and decide who should talk to whom. The dataset is about **7 million tokens**. No model reads that in one go.

What Entangle does. A team of agents reads the data in parallel and produces, for each company challenge:

- a **triage**: is it a research problem, an engineering service, or not research at all (hiring, a licence, a funding round);
- the **researchers whose own records fit** the problem, and one **pair** chosen with reasons and risks;
- a **proposal** for that pair: an industrial PhD topic or a one-page joint project, shaped like the example in `docs/brief/`;
- up to three **open EU calls** and three **EU projects** that fit (the Funding Scout);
- and a clear **"no good match"** when there is none. That is a correct answer, not a failure.

A **Critic** checks every claim against the record it cites, and a **person approves** each pair before anyone is contacted.

The one sentence to remember: *the agents write, a separate Critic checks every claim against the source it cites, and a human decides.*

The 60-second version, out loud

> The EINSTEIN dataset is about seven million tokens, which no model can read at once. So we split the work across a team of agents. Each one reads only one company or one researcher, and they all run at the same time on one Mozaik runtime. Each fact an agent writes carries the id of the record it comes from. A separate Critic agent, in its own context, checks every claim against that record, and sends back what does not hold. In our reference run it rejected 483 claims that would otherwise have reached a company. At the end a coordinator approves each pair. Nothing is sent to anyone before that.

The numbers worth knowing (reference run `out/golden/`)

WHAT	VALUE
Companies, challenges ("needs")	8 companies (C001 to C008), 29 needs
Researchers searched	all 1,047, with their 10,779 publications, projects and patents
Profiles written (passed the Critic)	66
Pairs (passed) / "no match" outcomes	32 / 7
Pairs chosen for a proposal	7
Proposals accepted / left for a person	6 / 1
Model calls	696, by 20 agents
Tokens, cost	4,958,225 tokens, $16.90 at API prices
Critic: reviews, sent back, dropped, cut-and-kept	337, 194, 10, 20
Critic: claims rejected	483
Trace	7,105 events in `trace.jsonl`
Why it stopped at 66 minutes	the 60-minute budget stopped the last company (C008) before matching

Part 2. The architecture in five ideas

2.1 One runtime, one bus

Everything runs in **one Node process** on one **Mozaik** runtime (`@mozaik-ai/core` 4.1.0). Mozaik is the library the hackathon asks for. Think of it as a room with a loudspeaker:

- A **participant** is anyone in the room. There are two kinds. `createAgent` makes a participant that **calls a model**. `createHuman` makes one that **does not** (in Mozaik's words it is a "human", for us it is plain code).
- An **event** is something said over the loudspeaker (`sendEvent`). **Every participant hears every event.**
- A participant reacts through a **handler**: a `SituationSpecification` ("when an event like this happens") plus a `processor` ("do this").

The runtime is created once in `src/runtime/mozaik.ts` (`defineRuntime<MatchState>()`). It is eleven lines. Read it.

Agents never call each other. The Analyst does not call the Matchmaker. It writes its result on the blackboard and says `artifact.posted` . Whoever cares reacts. This is why you can add a participant (the Funding Scout was added late) without touching the others.

2.2 One blackboard

`MatchState` (`src/runtime/state.ts`) is a class with a handful of `Map` s: `needs` , `profiles` , `candidates` (the pairs), `noMatches` , `proposals` , `funding` , `verdicts` , `tasks` , and the `budget` . Keys are **dataset ids** (`R0001` , `C001-CH1` , a pair id). Agents read and write it directly. Because every artifact is keyed by a real id, "who is this about?" is never ambiguous.

2.3 Every fact is a `Claim` with sources

```
interface Claim { text: string; sourceIds: string[] }   // src/types.ts
```

A claim is one short sentence plus the ids that support it: `R0001` , `C001-CH1` , `SYN-PUB-00001` , `SYN-PRJ-0001` , `SYN-PAT-001` , or a company page. **This is the design decision that makes checking possible.** The Critic never asks "is this true?" in general. It asks "does *this cited record* say this?". Models return claims through `structuredOutput` (a JSON schema), so there is no free text to parse for the facts.

2.4 Work is "tasks", and every task is its own loop

A **task** (`createTask` in `src/runtime/tasks.ts`) is one unit of work: `analyze` , `profile` , `match` , `write` , `critique` , `translate` , `scout` , `screen` . `runTask()` starts it as its own **Mozaik loop** with a **fresh context** (`runLoop`). Three consequences:

- One agent can have many loops open at once, so one Profiler can profile several researchers in parallel.
- Tasks never share memory. A call sees only the input for its own task. That is why the biggest call in the run read 19,447 tokens, not 7 million.
- A task that fails fails alone. A bad answer gets retried (up to `MAX_ATTEMPTS` , 3), then the task is marked failed and the rest goes on.

A **task tracker** (`createTaskTracker()` , also in `tasks.ts`) listens to the bus and keeps tasks, token counts, cost and the budget up to date. It marks a task done when the model's final `model.answer` arrives.

2.5 Nothing waits for a "stage"

There is no "phase 1, then phase 2". The **Orchestrator** hands out work **the moment its input exists**:

1. Every company goes to an Analyst at once.
2. As soon as one need is posted, the **Shortlist** finds candidate researchers, and Profilers start reading them, while other companies are still being read.
3. When a need's candidates are profiled, the need goes to the **Matchmaker**.
4. When every review of a company has ended, the Orchestrator **decides** that company's pair and gives it to a Writer. The **Funding Scout** hears the same decision and works at the same time.

5. The **Critic** reviews every artifact as it lands, beside all of this.

Result: in the reference run there were up to 5 roles calling a model at the same moment, and 2 or more for 48% of the time.

Part 3. The team, role by role

Each row names the folder to open (all under `src/agents/`). Every agent folder has the same shape: `<role>.ts` (the agent and its handlers), `prompt.ts` (the instruction and the JSON schema), `mock.ts` (canned answers for runs with no keys).

ROLE	COUNT	MODEL	CALLS A MODEL?	READS	WRITES ON THE BLACKBOARD	FOLDER
Orchestrator	1	none	no	events	assignments, the chosen pair per company	`orchestrator/`
Industry Analyst	4	Haiku 4.5	yes	one company (profile, challenges, pages)	one **need** per challenge	`analyst/`
Shortlist	1	none	no (a small decision model with `SCREEN=1`)	a need	up to 4 candidate researchers, by BM25 search	`profiler/shortlist.ts`, `pool.ts`
Research Profiler	6	Haiku 4.5	yes	one researcher and their records	one **profile**	`profiler/`
Matchmaker	1	Sonnet 4.6	yes	one need and its candidate profiles	**pairs** with fit 0 to 1, or a **no-match**	`matchmaker/`
Proposal Writer	3	Sonnet 4.6	yes	one chosen pair and its records	one **proposal**	`writer/`
Critic and **Critic 2**	2	Sonnet 4.6, and Haiku 4.5 (or SubQ)	yes	every artifact, as posted	verdicts, send-backs	`critic/`
Funding Scout	1	Haiku 4.5	yes	each chosen pair, plus the EU portal and CORDIS	up to 3 calls and 3 EU projects per pair	`scout/`

That makes 21 participants on the bus in the reference run (4 + 6 + 3 + 2 agents, the Matchmaker and the Scout, the Orchestrator, the Shortlist, the trace observer and the task tracker). Models are chosen per role in `config/models.ts`: cheap Haiku for the many parallel readers, Sonnet where quality is decided (Matchmaker, Writer, Critic).

What each role actually does

Industry Analyst. Reads one company and writes one **need** per challenge: the **triage** (`research`, an engineering service, or `not-research`) with the reason; an **English academic restatement** of the challenge (this is also how the 26 Serbian-language companies become searchable); **5 to 10 search terms** in

research vocabulary; what the company *brings* (data, lab, line, money); and its *constraints*. The search terms matter most: they are the query that finds researchers. See `applyAnalysis()` in `analyst.ts`: it keeps only claims whose ids really belong to this company (`ClaimFilter`).

Shortlist (no model). Takes the need's search terms (twice) plus the restatement and runs a **BM25** search over all researchers' publications, projects, patents and profiles (see Part 6). It skips people who left the university and retired professors who do not consult. `shortlistFor()` in `src/agents/profiler/shortlist.ts`. A "not research" need gets no candidates, and no model call at all.

Research Profiler. Reads one researcher's whole bundle (profile, publications, projects, patents) and writes a **profile**: expertise claims, industry-fit claims, and availability. Two safety rules are in code, not in the prompt: the identity fields (id, name, title, `canLead`) come from the data and never from the model (`identityOf()`), and the availability claim is checked against a list of personal words (parental, health, children…) and replaced by the dataset's already-redacted sentence if it leaks a reason (`availabilityOf()`). A researcher is profiled **once per run**, however many needs shortlist them (`pool.profile()`).

Matchmaker. Gets one need and its candidate profiles. Its prompt makes it first write the **core difficulty** (what the company cannot solve itself, plus its hard limits), then judge each candidate against that, not against the topic. It returns up to 3 pairs with a **fit** (0 to 1, for *this* pair and *this* problem), the problem the two would solve, reasons, and risks, or `no-match` with a reason. In code (`applyMatch()`): a pair under **0.5** is dropped, only candidates it was given can pair (an invented researcher id is dropped and logged as `match.dropped`), and if the researcher cannot lead, a risk is added automatically.

Orchestrator. `src/agents/orchestrator/orchestrator.ts`. The most useful file to read first. `decide(company)` picks a pair only when its **need, profile and pair all passed the Critic**, only above fit 0.5, and only for a researcher with no proposal yet in this run. Each researcher gets one proposal per run, and each company one, so the output is spread over many people. `choosePairs()` in `matchmaker/select.ts` compares fit **only within a challenge**.

Proposal Writer. Writes the proposal for a pair as a structured draft, which `render.ts` turns into Markdown. A PhD topic is allowed only if the researcher `canLead` **and** the need's triage is `research`; otherwise it must be a one-page joint project. With `OPENALEX_API_KEY`, the Writer first asks OpenAlex for recent real works on the need's topic (never a person's name), and may cite up to two.

Funding Scout. Two short model calls per chosen pair. The first writes search queries in the **EU's own vocabulary**. The **code** then queries the EU Funding & Tenders portal and CORDIS, and keeps only calls whose deadline is still ahead. The second call only *picks* from what the search returned. Every fact shown (title, dates, grant, link) is the source's own, not the model's.

Part 4. The Critic, in depth

This is the part the jury will push on, so know it well. Open `src/agents/critic/critic.ts` (reviews), `checks.ts` (the code checks), `prompt.ts` (what the model is told).

The Critic reacts to `artifact.posted` for **every** artifact (need, profile, pair, proposal). For each one it runs three steps.

Step 1. Code checks, no model (`checkByCode()` in `checks.ts`)

For every claim, in order:

CHECK	REJECTS WHEN	CODE NAME
Has a source	the claim cites no id	`no-source`
Source exists	a cited id is not in the dataset	`unknown-source`
Right subject	a cited record does not belong to this researcher or company. **By id, never by name**	`wrong-subject`
No borrowed details	a name or number is in a record of the same artifact that the claim does **not** cite	`uncited-detail`
No misquote	a Serbian term quoted in brackets is not in the cited record	`misquote`

Why by id, not by name: the data has traps. Two different people share a name, outside co-authors carry a researcher's exact name, and company staff share names with researchers. A name match would attach someone else's paper to a person. This check is cheap, deterministic, and catches a whole class of errors for free.

Step 2. Model check (`vote()`)

The claims that passed step 1 go to a model with the **cited records** attached. For each claim it answers whether the record says what the claim says, **with the reason first, then the verdict**. The rubric is `CRITIC_INSTRUCTION` in `prompt.ts`. It rejects, among other things: a wider scope than the record, a count widened ("all clients" when one asked), a judgement added to a fact ("directly addresses the company's problem"), and a word taken from a record the claim does not cite.

Three details that explain the numbers:

- **A re-posted artifact only has its changed claims re-checked.** Each reviewer remembers the claims it already accepted (`supported`).
- **Proposals get a second reviewer** (`dualReview` , default `["proposal"]`) on **another model** and in its own context, running at the same time. A claim passes only if **both** pass it. Disagreements are logged as `critic.split` (15 in the reference run).
- **A proposal's free text** (research questions, approach, plan, the reasoning) is *not* a claim. It gets its own check (`freeSentences()` , `TEXT_REVIEW_SCHEMA`): any sentence that states as true something no claim or source states is rejected as `free-text` . This was added after a hand-check found 20 invented facts in three proposals, all in the free text.

Step 3. Verdict (`decide()`)

OUTCOME	WHEN	WHAT HAPPENS
accept	no claim rejected	artifact counts as passed
send-back	something rejected, fewer than 2 send-backs so far	a **new task** goes to the producer with each rejected claim and its reason. The producer fixes **only those claims** (rewrite or drop) and re-posts
trim (`critic.trimmed`)	fixes ran out, and the failed claims are not "core"	the Critic **cuts** the failed claims and accepts the rest (`trimArtifact()`)
drop	fixes ran out, and a core claim failed	the artifact is removed (need, profile, pair)

OUTCOME	WHEN	WHAT HAPPENS
human review	same, for a proposal	the proposal goes to `needs-human/` for a person

"Core" means: a need's restatement, a pair's problem, a profile's availability, and a proposal's header and its last claim under "The problem" and "Why this pair". The trim rule exists because every drop in the earlier runs had exactly one claim left over.

The Orchestrator does **not** wait on the Critic directly. It waits for `inReview()` to be false, and only then calls `decide()`. A Writer never starts on a pair whose need, profile or pair is still in doubt.

What the Critic is and is not

- **Is:** a separate agent with its own context. It sees the claim and its cited records, **not** the writer's reasoning. That is why it is a real second opinion and not a model grading itself.
- **Is not perfect.** Measured on 272 hand-labelled claims (`test/fixtures/critic-labels.json`), the Critic catches **53 to 67%** of the wrong ones and wrongly rejects **3 to 6%** of the right ones. It misses subtle things: a widened scope, a judgement added to a fact.
- **Checks support, not quality.** It does not say a proposal is *good*. That comes from the prompts, and from the person who reads it.
- **A Haiku Critic was tested and rejected:** it caught 11 of 23 wrong claims where Sonnet caught 15 to 21. It would save about 38% of Critic cost, so Sonnet stays.

Part 5. The runtime plumbing

Everything here is in `src/runtime/`. It is what makes 700 parallel model calls behave.

5.1 Every call goes through one `ResilientRunner`

Mozaik lets you give it your own `InferenceRunner` (the thing that actually calls a model). Ours, `resilient-runner.ts`, wraps the real one and adds:

- **A global limit:** at most **16 calls in flight** (`MAX_IN_FLIGHT`), and no more than the number of local Claude Code sessions (6).
- **Rate limits per provider:** a token bucket for requests and input tokens per minute (`PROVIDER_LIMITS` in `config/models.ts`).
- **Priority:** a freed slot goes to the call **furthest down the pipeline** (`RANK`: writes and proposal reviews first, analyses last), so a company that is almost done does not wait behind new companies' profiles.
- **Retries:** 429, 5xx and network errors are retried with exponential backoff and jitter, honouring `retry-after`. A 429 pauses the whole provider. Each retry is a trace event.
- **Stop rules:** no call goes out for a task that has already failed or timed out, or once the budget plus 10% slack is spent.

5.2 The budget

Four limits, from `config/models.ts` and `.env`: tokens (6M), dollars ($20), wall clock (45 min), attempts per task (3). `budgetSpent()` in `tasks.ts` checks them. At the limit **no new task starts**. Running tasks get 10% more to finish. The reference run used `BUDGET_USD=40 BUDGET_MINUTES=60 BUDGET_TOKENS=12000000`.

5.3 Four backends behind the same interface

BACKEND	WHEN	FILE
API (Anthropic, OpenAI, Gemini)	a key is set in `.env`	Mozaik's own runner, via `defaultInferenceRunner()`
Local Claude Code	`LOCAL_CLAUDE=1` or `--local`	`claude-code-runner.ts`: each call is a headless `claude -p` session on your own login, no API key
Mock	`--mock`, or no key at all	`mock-runner.ts`: fake latency, canned answers from each agent's `mock.ts`
Decision model (SubQ, Jev)	`SUBQ=1` / `SCREEN=1`	`snap-runner.ts`: answers one yes/no question per claim or candidate, returns no text

Because they share one interface, a run is the **same code** on all four. The mock plants one invented claim in a need, a profile and a proposal so you can **watch the Critic catch each one** with no keys and no cost.

5.4 The trace

`src/trace/trace-observer.ts` is a participant that hears **every** event and writes one JSON line each to `out/<run-id>/trace.jsonl` (`kind`, `ms`, `agent`, `taskId`, `loopId`, `model`, tokens, cost). Event kinds you will see:

`task.started`, `inference.dispatched` (the runner let a call out; the time before it is a wait), `inference.completed`, `task.completed`, `artifact.posted`, `critic.reviewing`, `critic.verdict`, `critic.sent_back`, `critic.trimmed`, `critic.dropped`, `critic.split`, `shortlist.ready`, `match.none`, `orchestrator.decided`, `funding.posted`, `inference.retry`, `task.failed`, `budget.exhausted`.

The **web app is built on this one file.** It replays the trace. That is why a run folder is self-contained and opens with no keys.

Part 6. The data layer

All in `src/data/`. **No model calls anywhere in this folder.**

6.1 Loading and cleaning (`dataset.ts`, `normalize.ts`)

Before any agent reads a record, deterministic code cleans it:

PROBLEM IN THE DATA	WHAT WE DO
4 duplicate researcher profiles (R0139, R0556, R0614, R0894)	merged into their main profile on load (`DUPLICATES`). A researcher's `aliases` keep the old ids

PROBLEM IN THE DATA	WHAT WE DO
Faculty, title, programme spelled 37, 26, 22 ways, in English, Latin and Cyrillic Serbian	mapped to one canonical English form (`canonFaculty` , `canonTitle` …)
Emeritus and departed researchers	`status` is set. They can never be proposed to lead or supervise (`canLead` false). Departed ones are never proposed at all
Personal reasons for unavailability (parental leave, children, health)	removed on load (`redactPersonal`). Only the timing stays: "on leave until September 2026"
Emails on company pages	masked before an agent reads them (`maskEmails`)
Two people with one name	stay two people. Identity is **by id only**

6.2 Search: BM25, not embeddings

`bm25.ts` is a ~65-line implementation of Okapi BM25 (k1 = 1.2, b = 0.75). It needs no API key, runs in memory and is deterministic. The tokenizer (`text.ts` , `analyze()`) folds **Cyrillic to Latin**, strips diacritics, lowercases, removes stop words and applies a **Porter stemmer**, so `Марковић` and `Markovic` match.

`findResearchersForNeed()` in `search.ts` is the heart of retrieval:

1. Search one index of all researchers' records (publications, projects, patents, one profile document per researcher): top 1,500 hits.
2. Group the hits by researcher.
3. A researcher's score is the **mean of their 3 best-matching records** (missing ones count 0). **Not the sum.**
4. Scores are made relative to the best match for this query (`relevance` 0 to 1).

Why the mean of the top 3 and not the sum? A sum rewards whoever has the most papers (0 to 23 in the data), which is close to ranking people by output. The mean of three means a few strong records beat many weak ones. It is also a **responsible-data decision**: we compare fit to a problem, never productivity.

Why queries come from the Analyst, not from the raw challenge text (`retrievalText()`): companies write in business language, and 26 of them in Serbian. The Analyst's English academic restatement and search terms bridge that gap. This is where a model earns its place in retrieval.

6.3 The reverse search (`problem.ts`)

A company that is not in the dataset types one problem in its own words (`--problem "..."` , or **Describe a problem** in the app). The text becomes a company record `C999` with one challenge `C999-CH1` that holds nothing else: no size, products or pages are invented. After that it is the **normal pipeline** on one challenge, and the Critic checks claims against the typed text.

6.4 Your own data (`import.ts`)

The app's **New run** accepts files in the dataset's format. `import.ts` reads them leniently (a missing field is empty), and gives foreign ids the dataset's shape (`jdoe` becomes `R0001`) because the Critic and the app find cited ids **by that shape**. Uploads stay on the machine in `data/uploads/<name>/` , and a run on them writes `dataset.json` so the app looks up its sources there.

6.5 External sources (`openalex.ts` , `eucalls.ts` , `cordis.ts`)

All three are searched **by topic words only, never by a person's name** (a synthetic name would match real people). Every fact from them is saved in the run's `sources.json`, so the run replays with no network. OpenAlex works are cited as `W...` ids, and the Critic checks that such an id exists **only if OpenAlex returned it in this run**, so an invented paper fails at the code check.

Part 7. The translator (a detail people ask about)

26 companies and about 82 researchers write in Serbian. A model checking "bark damage" against "oštećenja kore" may translate the word the *claim* chose ("kora" as bark) instead of the word the *sentence* means. So `src/agents/critic/translate.ts` makes **one English translation of each Serbian line per record per run**, as its own cheap `translate` task. The Analyst and the Writer write from it, and the Critic shows it under the original when it checks. All three read **the same English**. `isSerbian()` detects Serbian lines with word lists (no model).

Part 8. The web app and the server

- **Server** (`src/server/`): a small **Hono** app. `main.ts` starts it on port 4317 (or the next free one), bound to `127.0.0.1` only because the API starts processes and writes files. `spawn.ts` (`startRun`) starts the same CLI (`src/cli.ts`) as a **child process** when you press start in the app, and `runs.ts` (`RunRegistry`) keeps the list of run folders in `out/` . The app never runs agents itself: it only reads the trace the child writes. `app.ts` has the routes: `/api/runs` , `/api/runs/:id/trace` , `/api/runs/:id/events` (a server-sent-events stream of the trace lines, resumable with `Last-Event-ID`), `/api/runs/:id/results` , `/approvals` , `/api/datasets` .
- **Web app** (`web/`): React 19, Vite, Tailwind 4. Read `web/README.md` . The key idea: **one store folds the trace** (`web/src/run/`) into agents, loops, tasks and totals. Four screens read it: **Stage** (the agents round the event bus), **Timeline** (a lane per agent, a bar per model call), **Overview**, **Results** (pairs and proposals, with every cited source one click away, plus approve or reject).
- **Approvals** are written to `out/<run>/approvals.json` by the server. In the static GitHub Pages build they stay in the browser.

Useful commands:

```
npm run replay -- out/golden      # the reference run in the app, no keys, no dataset
npm run match -- --mock           # the whole team on the mock model, free
npm run dev                       # the app with live runs and hot reload
npm run typecheck                 # tsc for the server and the web app
```

Part 9. Honest limits (say these before they ask)

1. **The data is synthetic.** Real use needs real profiles and the people's consent.

2. **Retrieval is BM25 on the Analyst's terms.** A researcher who describes the same work in other words can be missed, and only 4 candidates per need (2 with `SCREEN=1`) are profiled. Next step: multilingual embeddings fused with BM25.
3. **The Critic is not perfect** (53 to 67% of wrong claims caught, 3 to 6% of right ones rejected). It checks *support*, not *quality*. That is why a person approves every pair.
4. **We did not run a single big model as a baseline.** The dataset does not fit in one call, and with retrieval added it is already a pipeline. What we *did* measure is how many wrong claims a strong model writes unchecked: in a hand-check of three proposals, **15 of 76 claims (20%)** were wrong, and the first Critic had passed them.
5. **Mozaik 4.1.0 cannot cancel a loop.** A task stopped by the budget is marked failed and its loop is *dropped* (a promise that never resolves), not killed (`dropped()` in `resilient-runner.ts`). It also sends no `cache_control`, so prompt caching on the API would need a wrapper.
6. **Local Claude Code runs 6 sessions at once,** so a run on it is slower than on the API. In the reference run, 11 `claude -p` calls hung for 5 minutes each before the budget ended the run.
7. **The reference run covers 8 of 153 companies.** A run on all of them is about 69M tokens and $250. That is a cost decision, not a technical limit.
8. **One proposal per company and per researcher per run,** by design.
9. **Known failing unit test:** `test/stage-words.test.ts` fails when run with plain `tsx` because the web code imports through the `@/` alias, which only Vite resolves. It does not affect the app or a run.

Part 10. Questions we should expect

"Why a team of agents and not one big model?" The data is 7M tokens and does not fit in one call. Each call here reads one company, one researcher or one pair: the median call read 6,285 tokens, the largest 19,447. Second, a model does not check itself: our Critic is a separate agent that sees only the claim and its cited records. Third, we can put the right model on each job. Full answer in `docs/qa.md` (in Serbian) and the README section *Why a team of agents*.

"Is this real parallelism or just async calls?" Real. Each task is its own Mozaik loop, an agent can have several open, and the runner lets 16 calls run at once (6 with local Claude Code). Up to 5 roles called a model at the same moment, 2 or more for 48% of the run. Open the Timeline (`npm run replay -- out/golden`): overlapping bars are the proof.

"How do you stop the model from making things up?" Four layers. (1) Models return claims with `sourceIds` as structured output. (2) Code strips ids that do not belong to the subject before the artifact is even posted (`ClaimFilter`). (3) The Critic's code check rejects unknown, foreign or uncited-detail claims. (4) A model reads the cited records and judges support, with a second reviewer for proposals. Anything left goes to a person.

"What does the Critic *not* catch?" Subtle things: a widened scope, a judgement added to a fact. It catches 53 to 67% of wrong claims on our labelled set. That is why a human approves each pair.

"How do you pick researchers? Is it ranking people?" No. It is a search for records that fit **one stated problem**. BM25 on the Analyst's terms, then a per-researcher score that is the mean of their 3 best records (not a sum, so volume does not matter). Fit is compared only **within a challenge**. The system never outputs a score or a list of people. The Matchmaker prompt forbids seniority, publication counts and prestige as reasons.

"What happens when a challenge is not a research problem?" The Analyst triages it as `not-research`, and the Matchmaker returns a no-match with **no model call** (`settleWithoutModel()` in `matchmaker.ts`). In the reference run, 7 needs ended as no-match.

"How do you handle the data traps (duplicates, shared names, stale profiles)?" In code, at load time: duplicates merged, labels normalised, emeritus and departed flagged, personal reasons redacted. Identity is by id only. Details in `docs/dataset.md`.

"What about privacy?" Only the provided dataset, plus published work **by topic**. No agent searches any API for a person. Personal reasons for availability are removed on load, emails are masked, notes are used as constraints and never quoted, and a person approves before any outreach.

"How do you handle rate limits and failures?" `ResilientRunner`: 16 calls at once, a token bucket per provider, retries with backoff and jitter, `retry-after` honoured, a 429 pauses the provider. A failed task fails alone. In the reference run there were 11 retries and 19 failed tasks, and the run still produced 6 accepted proposals.

"How much does it cost?" The reference run (8 companies): 4.96M tokens, $16.90 at API prices. Re-reviews are about 26% of the cost. All 153 companies would be about 69M tokens and $250.

"Why Mozaik?" The hackathon asks for it, and it fits: an event bus where every participant sees every event gave us a Critic that reacts the moment something is posted, and a trace for free. Our own `InferenceRunner` gave us rate limits, retries, budget and a mock model in one place.

"What did Mozaik not give you?" It cannot cancel a loop (we fail the task and drop the loop), and it sends no `cache_control`.

"Can I see it work with no keys?" Yes. `npm run replay -- out/golden` shows the real reference run. `npm run match -- --mock` runs the whole team on the mock model and plants invented claims for the Critic to catch.

"How would this plug into EINSTEIN for real?" Only `src/data/` changes: the agents work on ids. Researcher side from OpenAlex, ORCID and CORDIS by id (never by name), company side from a short challenge form. Coordinators at UNS and NTP approve pairs, and EINSTEIN makes the introduction. See the README section *How this plugs into EINSTEIN*.

"Who wrote this code?" It was built with an AI coding assistant. What we can stand behind is the design (Parts 2 to 6), the checks (Part 4) and the measured results (`docs/quality.md`). When a question goes past this document, open the file named at the end of each part and read the function. Most of the files are short.

Part 11. Where to look: a map of the repository

YOU WANT TO EXPLAIN...	OPEN
The runtime and the bus	`src/runtime/mozaik.ts` (11 lines), `src/runtime/index.ts` (`startRuntime`)
The blackboard	`src/runtime/state.ts`
Tasks, loops, the budget	`src/runtime/tasks.ts` (`createTask`, `runTask`, `whenSettled`, `budgetSpent`)
Rate limits, retries, priority	`src/runtime/resilient-runner.ts`, `config/models.ts`
Which model does what	`config/models.ts` (`ROUTES`)
The whole run, step by step	`src/agents/orchestrator/orchestrator.ts` (`onShortlist`, `ready`, `decide`, `settle`)

YOU WANT TO EXPLAIN...	OPEN
How a run starts	`src/cli.ts` (`main`)
Where a run's files are written	`src/agents/orchestrator/output.ts` (`writeRun`)
The Critic	`src/agents/critic/critic.ts` (`reviewOnce` , `vote` , `decide`), `checks.ts` (`checkByCode` , `trimArtifact`), `prompt.ts`
How the Critic talks to producers	`src/agents/critic/protocol.ts` (`postArtifact` , `SentBackToMe`)
How an agent is built	`src/agents/profiler/profiler.ts` (the smallest complete example)
Retrieval	`src/data/bm25.ts` , `src/data/search.ts` , `src/agents/profiler/shortlist.ts`
Data clean-up	`src/data/normalize.ts` , `src/data/dataset.ts`
The reverse search	`src/data/problem.ts`
The trace	`src/trace/trace-observer.ts`
The app	`web/README.md` , `web/src/run/` , `web/src/screens/`
Measured quality	`docs/quality.md` ; dataset traps: `docs/dataset.md` ; answers to other teams: `docs/qa.md`

A 20-minute self-test

Do these once. After them you can answer most "how does it really work" questions from experience, not from this page.

1. `npm run replay -- out/golden` , press play on the Stage, then open the Timeline. Find a bar with an orange mark: a send-back.
2. `grep '"critic.sent_back"' out/golden/trace.jsonl | head -1` , and read the `rejected` list: claim, reason, which check caught it.
3. Open `src/agents/orchestrator/orchestrator.ts` , find `decide()` , and read the list of reasons a company can end with **no proposal**.
4. Open `src/agents/critic/checks.ts` , find `checkByCode()` , and name the five checks from memory.
5. `npm run match -- --mock` , and watch the CLI: find the planted claims being sent back and fixed.
6. Open `config/models.ts` , and say which role runs on which model and why.